LAB 4 — VIBE CODING

Mastermind Game
Prompts & Antigravity Responses

A consolidated record of the prompts used for the assignment and the corresponding implementation summaries supplied by Antigravity.

Project	20_mastermind — Python terminal-based Mastermind game
AI coding tool	Antigravity
Tasks covered	Task 1: Feedback logic; Task 2: Game lifecycle; Task 3: Difficulty modes; Task 4: History and input robustness; README update
Source	User-provided prompts and Antigravity response text in this conversation

Document note: The prompts below are reproduced from the earlier guidance, and the responses are organized from the Antigravity output supplied by the user. Reported test results are presented as Antigravity reported them; this document does not independently execute or verify the project.

How to read this document

Each task contains the prompt to give the coding assistant, followed by a structured version of the corresponding Antigravity response. Code and test results are included where they were supplied. File links originally shown as local file:// paths are described by filename because those links point to the user's local computer and are not accessible from this PDF's reader.

Prompt 1 — Inspect the Mastermind Project

> I am completing an individual Python Vibe Coding lab. The assigned project is a terminal-based Mastermind game.
>
> Inspect the existing README.md, main.py, game.py, logic.py, and requirements.txt.
>
> Before changing anything, explain:
> 1. The purpose of each file.
> 2. How the game currently works.
> 3. The current feedback-calculation bug.
> 4. Which features required by the README are missing or incomplete.
> 5. The test cases needed to verify the fix and all required features.
>
> Do not modify the code yet. Do not replace the project with an unrelated implementation. Keep the existing project structure and standard-library-only approach unless a dependency is genuinely necessary.

Antigravity response — Initial inspection

Antigravity inspected README.md, main.py, game.py, logic.py, and requirements.txt before making changes. It described the project structure and identified the main defect and the four required tasks.

File	Purpose reported by Antigravity
main.py	Entry point; imports Mastermind and starts the interactive session.
game.py	Game controller and terminal loop; manages secret code, turns, history, validation, feedback, and end conditions.
logic.py	Scoring engine; computes exact and partial matches.
requirements.txt	Declares that the project uses Python's standard library, with no external dependencies.
README.md	Lab specification, required tasks, tests, and deliverables.

Bug identified

The original partial-match calculation used sum(ch in code for ch in guess) - exact. Membership checks do not consume symbol occurrences, so repeated symbols in the guess can be counted multiple times even if the secret code contains only one such symbol.

```
Original problematic logic:
exact = sum(a == b for a, b in zip(code, guess))
partial = sum(ch in code for ch in guess) - exact
return exact, partial
```

Example: code ['1','2','3','4'], guess ['1','1','1','1']. The old implementation reports (1, 3), while the correct result is (1, 0).

Missing or incomplete requirements

• **Task 1:** Duplicate-aware feedback, with exact matches resolved first.

• **Task 2:** Clear lifecycle state, win/loss handling, guess limits, and protection against post-game state changes.

• **Task 3:** Difficulty modes changing code length, symbol range, and allowed guesses.

• **Task 4:** Readable guess history and robust input validation that does not consume turns for invalid guesses.

Tests identified included exact matches, no matches, permutations, repeated symbols, mixed exact/partial matches, arbitrary lengths, wins, losses, quitting, invalid input, and difficulty-specific validation.

Task 1 — Correct duplicate-aware feedback

Implement Task 1 from the existing README by correcting logic.py.

The current implementation can overcount partial matches when symbols are repeated.

Requirements:
1. Exact matches must be counted first.
2. An exact match must consume its corresponding occurrence in the secret code.
3. Partial matches must only use unmatched secret-code occurrences.
4. Each secret-code occurrence may contribute to feedback at most once.
5. Preserve the existing feedback(code, guess) interface unless a change is necessary and justified.
6. Do not hard-code the original four-symbol configuration.
7. Do not add third-party dependencies.

Add or propose tests for:
- A completely correct guess.
- A guess with no matches.
- All matches in the wrong positions.
- Repeated symbols in the secret code and guess.
- A mixture of exact and partial matches.

Explain the algorithm and show the expected output of the tests. Do not silently rewrite unrelated files.

Antigravity response

Antigravity reported that it edited logic.py, created test_logic.py, ran the tests, and checked Git status.

Algorithm reported

• **Pass 1:** Compare each code/guess position. Count exact matches and collect only the symbols from non-exact positions.

• **Pass 2:** Count symbol frequencies in the unmatched code and guess using collections.Counter. For each symbol, add min(code_count, guess_count) to the partial count.

• The function keeps the feedback(code, guess) interface, supports arbitrary lengths and symbol alphabets, and uses only the Python standard library.

```
from collections import Counter

def feedback(code, guess):
    exact = 0
    unmatched_code = []
    unmatched_guess = []

    for c, g in zip(code, guess):
        if c == g:
            exact += 1
        else:
            unmatched_code.append(c)
            unmatched_guess.append(g)

    code_counts = Counter(unmatched_code)
    guess_counts = Counter(unmatched_guess)
    partial = sum(
        min(code_counts[sym], guess_counts[sym])
        for sym in guess_counts
    )
    return exact, partial
```

Reported test results

Test case	Code	Guess	Expected fixed result
Completely correct	1234	1234	(4, 0)
No matches	1234	5656	(0, 0)
All wrong positions	1234	4321	(0, 4)
Repeated guess symbols	1234	1111	(1, 0)
Repeated guess symbols	1234	2526	(0, 1)
Repeated code symbol	1123	4516	(0, 1)
Mixed exact/partial	1122	1212	(2, 2)
Excess repeated symbol	1123	1112	(2, 1)
Arbitrary configuration	ABCDE	ACBED	(1, 4)

```
Reported command:
python test_logic.py

Reported output:
.......
----------------------------------------------------------------------
Ran 7 tests in 0.000s

OK
```

Note: Antigravity's supplied summary says the suite ran 7 tests, while its table lists 9 scenarios. The PDF preserves that reported information rather than attempting to infer which scenarios were implemented as individual tests.

Task 2 — Complete game lifecycle

Implement Task 2 from the existing README in game.py while preserving the current project structure.

Requirements:
1. Provide clear win and loss handling.
2. Preserve the history of accepted guesses and their feedback.
3. Enforce the maximum number of guesses.
4. Prevent game-state changes after the game has ended.
5. Reveal the secret code when the player loses.
6. Allow the user to quit cleanly.
7. Keep the game state in memory; do not add files, databases, or persistent storage.
8. Preserve the corrected feedback algorithm.

Explain each important change and provide tests or manual test instructions for a win, a loss, quitting, and attempts to change state after the game ends.

Do not replace the project with an unrelated implementation.

Antigravity response

Antigravity reported editing game.py, creating test_game.py, and running both the direct test script and unittest discovery.

Changes reported

• Added is_finished and is_won flags, max_turns, and an in-memory history of accepted guesses.

• Added submit_guess(raw), which rejects guesses after game completion, validates input, records accepted feedback, and decrements turns only for valid guesses.

• A correct guess sets the win and finished flags. Running out of turns marks the game finished.

• Win messaging includes the number of turns taken; loss messaging reveals the secret code.

• Added clean quitting through q or quit. Re-running a finished game displays a notification and exits.

• Added optional code and max_turns parameters to support deterministic testing while keeping Mastermind().run() compatible with main.py.

```
Representative lifecycle state and method interface reported:
Mastermind(code=None, max_turns=10)
submit_guess(raw)
quit()
run()

Reported post-game guard:
if self.is_finished:
    raise RuntimeError("Cannot make a guess: game has already ended.")
```

Reported automated test results

```
python test_game.py
..........
----------------------------------------------------------------------
Ran 11 tests in 0.002s

OK

python -m unittest discover
..................
----------------------------------------------------------------------
Ran 18 tests in 0.003s

OK
```

Manual tests supplied by Antigravity

```
# Deterministic win test
python -c "from game import Mastermind; Mastermind(code=['1', '2', '3', '4']).run()"
# Enter 1234; expect a win message.

# Loss test
python -c "from game import Mastermind; Mastermind(code=['1', '2', '3', '4'], max_turns=2).run()"
# Enter 5555, then 6666; expect loss message and secret code.

# Quit test
python main.py
# Enter q; expect clean exit.

# Post-game guard test
python -c "from game import Mastermind; g = Mastermind(code=['1', '2', '3', '4']); g.submit_guess('1234'); g.submit_guess('1
# Second call is expected to raise RuntimeError.
```

Task 3 — Difficulty modes

Implement Task 3 from the existing README.

Add clearly documented difficulty choices that change all three of the following:
- Secret-code length.
- Permitted symbol range.
- Number of allowed guesses.

Requirements:
1. Make the actual game configuration depend on the selected difficulty.
2. Validate guesses using the selected code length and symbol range.
3. Keep the duplicate-aware feedback algorithm generic so it works with every supported configuration.
4. Ensure the displayed remaining-turn count corresponds to the selected difficulty.
5. Keep the existing terminal-based game and Python standard-library-only approach.
6. Document the supported difficulty configurations.

Explain the design and provide tests or manual test instructions for every difficulty. Do not introduce persistent storage or replace the project with an unrelated application.

Antigravity response

Antigravity reported that it implemented difficulty modes in game.py, documented them in README.md, and added automated coverage to test_game.py.

Difficulty configurations reported

Mode	Code length	Symbols	Allowed guesses
Easy (easy/e)	3	1–4	12
Medium (medium/m)	4	1–6	10
Hard (hard/h)	5	1–8	8

Design details reported

• A centralized DIFFICULTIES configuration stores code length, symbols, display range, turn limit, and description.

• Mastermind(difficulty=...) accepts easy, medium, or hard; Medium is the default for compatibility.

• Interactive startup presents a difficulty selection menu. Enter or m selects Medium; e and h select Easy and Hard.

• Guess validation uses the active mode's code length and allowed symbols.

• The feedback function remains generic and works for supported code lengths 3, 4, and 5.

Reported test results

```
python -m unittest discover
.......................
----------------------------------------------------------------------
Ran 23 tests in 0.004s

OK
```

Manual testing examples supplied by Antigravity

```
python -c "from game import Mastermind; Mastermind('easy').run(prompt_difficulty=False)"
# Easy: 3 digits, 1–4, 12 turns.

python -c "from game import Mastermind; Mastermind('medium').run(prompt_difficulty=False)"
# Medium: 4 digits, 1–6, 10 turns.

python -c "from game import Mastermind; Mastermind('hard').run(prompt_difficulty=False)"
# Hard: 5 digits, 1–8, 8 turns.
```

Task 4 — History and input robustness

Implement Task 4 from the existing README.

Requirements:
1. Display a readable history of accepted guesses with exact and partial feedback.
2. Reject malformed guesses with a clear error message.
3. Invalid guesses must not consume turns.
4. Invalid guesses must not be added to the accepted-guess history.
5. Validate guesses according to the selected difficulty.
6. Ensure feedback is calculated for exactly one accepted guess at a time.
7. Support the documented quitting behaviour.
8. Keep the implementation understandable and modular.

Preserve the existing game functionality, the corrected duplicate-aware feedback, and the difficulty settings.

Explain the changes and provide tests or manual test instructions for invalid lengths, invalid symbols, valid guesses, history display, and quitting.

Antigravity response

Antigravity reported implementing Task 4 in game.py and expanding test_game.py. It also reported that the complete suite contained 27 tests and passed.

Changes reported

• Added validate_guess(raw), returning (is_valid, error_message). It handles empty input, wrong length, and invalid symbols.

• Validation adapts to the active difficulty and provides specific error messages.

• Invalid input does not decrement turns, append to history, or call feedback().

• Added format_history() and display_history() to present an aligned table containing turn number, guess, exact count, and partial count.

• History is shown during gameplay, at game conclusion, and on demand with h or history.

• submit_guess(raw) calls feedback exactly once for each accepted guess.

• The q and quit commands retain the clean quitting behaviour.

```
Representative history layout reported:
--- Guess History ---
  #    | Guess | Exact | Partial
  -----+-------+-------+--------
  1    | 1234  | 1     | 2
  2    | 1256  | 2     | 0
--------------------
```

Reported automated test results

```
python -m unittest discover
.........................
----------------------------------------------------------------------
Ran 27 tests in 0.003s

OK
```

Manual test cases supplied

Input/scenario	Expected behaviour
Too short: 12	Reject; no turn consumed; no history entry.
Too long: 123456 in Medium	Reject; no turn consumed.
Out-of-range digits: 1278 in Medium	Report invalid symbols; no turn consumed.
Letters: abcd	Report invalid symbols; no turn consumed.
Valid guess: 1234	Evaluate once, record feedback, decrement one turn.
h or history	Display history without consuming a turn.
q or quit	Exit cleanly and reveal the secret code, as reported.

Final review prompt

Review the complete implementation against the original README.

Check every requirement individually:
1. Duplicate-aware feedback, with exact matches resolved before partial matches.
2. Clear win and loss handling.
3. Guess-history preservation.
4. Guess limits and prevention of state changes after the game ends.
5. Difficulty modes that vary code length, symbol range, and guess limit.
6. Readable history.
7. Invalid input rejected without consuming turns.
8. Correct feedback for one accepted guess at a time.
9. Exact matches, repeated symbols, no matches, mixed exact/partial cases, every difficulty, invalid guesses, final-turn wins, losses, and quitting.

Find genuine bugs and make only the necessary changes. Do not claim that tests passed unless they were actually executed and passed.

Preserve the original project structure, avoid unnecessary third-party dependencies, do not add persistent storage, and do not replace the game with an unrelated implementation.

At the end, provide:
- A checklist of implemented requirements.
- The actual tests executed and their results.
- Any remaining limitations.
- The exact commands required to run and test the game.

Antigravity activity and final review summary

The supplied activity log shows that Antigravity viewed and edited game.py and test_game.py, ran python -m unittest discover multiple times, exercised main.py with simulated input, checked README.md, and updated the README. The final supplied summary says the README documents the completed implementation.

README sections reported as updated

• Game purpose and rules for exact and partial feedback.

• Setup and run commands, including Python standard-library-only requirements.

• Easy, Medium, and Hard configurations.

• Duplicate-aware feedback algorithm.

• Input validation and guess history behaviour.

• Commands for running the full test suite and individual test modules.

• Checklist of Tasks 1–4 and known limitations, including a terminal-only interface and in-memory state.

Reported final test status: 27 tests passed with python -m unittest discover. This is a report of the supplied Antigravity output, not an independent verification by the author of this PDF.

README update prompt

Update the existing README to document the completed Mastermind implementation.

Preserve the original project's identity and assignment context.

Document:
- The purpose of the game.
- The setup and run commands.
- The difficulty modes and their actual configurations.
- The rules for exact and partial feedback.
- Input validation and history behaviour.
- How to run the available tests.
- The key requirements implemented.
- Known limitations, if any.

Do not invent functionality that is not present in the code. Do not claim tests passed unless they were executed successfully. Do not add unrelated features or introduce unnecessary dependencies.

Antigravity response — README updated

Antigravity stated that README.md was updated to describe the completed Mastermind game while preserving the original lab context and project identity. It listed the following sections as added or updated:

• Game purpose and rules.

• Setup and run commands.

• Difficulty configurations and interactive startup behaviour.

• Exact and partial matching rules.

• Input validation and history display.

• Test commands for the reported 27-test suite.

• Completed task checklist and known limitations.

Consolidated final reported test command

```
python -m unittest discover
```

The Antigravity response reported 27 passing tests. Before submission, run this command in your actual project folder and check the output yourself.

Final implementation summary

Requirement	Implementation reported by Antigravity
Task 1 — Feedback	Two-pass matching; exact matches consume positions; Counter limits partial matches by symbol frequency.
Task 2 — Lifecycle	Win/loss state, turn limits, accepted-guess history, quitting, and post-game guard.
Task 3 — Difficulty	Easy: 3 symbols/1–4/12 turns; Medium: 4/1–6/10; Hard: 5/1–8/8.
Task 4 — Input/history	Mode-aware validation, no turn loss for invalid input, formatted history, h/history command.
Documentation	README updated with setup, rules, difficulty, tests, and limitations.
Testing	Antigravity reported 27 passing tests using unittest discovery.

Important verification note

This PDF organizes the prompts and the Antigravity responses supplied by the user. It does not establish that the final local files match every code excerpt or that the reported tests can be reproduced in the current environment. In particular, run the final tests, inspect git status, and verify the actual final code before submitting the lab.